JAVASCRIPT LIBRARY

What is React?

A powerful JavaScript library for building modern, dynamic user interfaces — without the overhead of full page reloads.

Single-Page Applications

React is purpose-built for SPAs — applications that load once and update dynamically as users interact, delivering a fast, app-like experience in the browser.

Performance by Design

By enabling dynamic UI updates without full page reloads, React dramatically improves perceived performance and responsiveness — keeping users engaged and reducing server load.

naresh.chaurasia@cognizant.com

Core React Features

React's design philosophy centres on simplicity, reusability, and developer productivity. These four pillars define how React applications are built.

Component-Based Architecture

UI is built from small, reusable components — each responsible for its own rendering and logic.

Declarative Views

Define *what* the UI should look like based on state — React handles the *how* automatically.

Logic in JavaScript

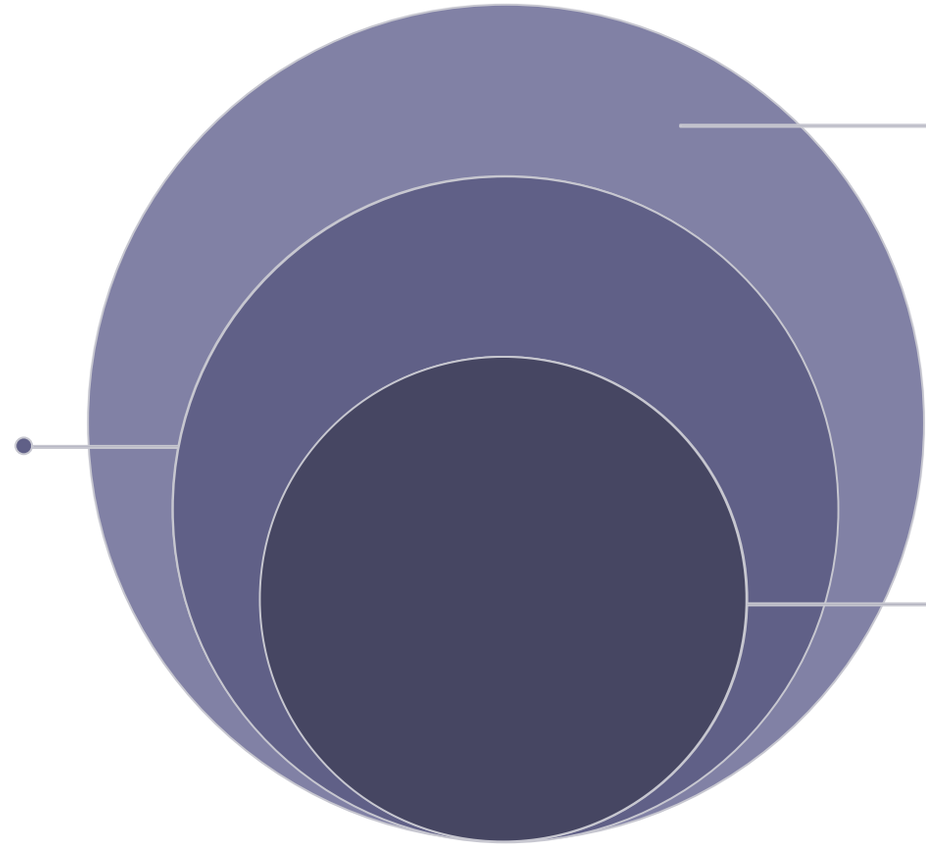
UI and behaviour live together in one place — no separation of concerns between markup and logic.

Modern UI Frameworks

Seamlessly integrates with Bootstrap, Material UI, and other popular design systems.

React Architecture

A React application is structured as a **hierarchy of components** — from the top-level app down through nested children. Understanding this tree is fundamental to building well-structured React apps.



Child Components

Render UI and trigger state updates

Parent Components

Manage sections and pass props down

App

Root component; manages global state

Each component in the hierarchy renders its own HTML via JSX and contains its own JavaScript logic. When a component's state changes, React efficiently re-renders only the affected parts of the UI — not the entire page.

Key Concepts

Mastering React means understanding five foundational concepts that work together to create dynamic, maintainable applications.

1	**Component** A reusable UI unit combining view and logic — the fundamental building block of every React application.
2	**Props** Short for *properties* — the mechanism for passing data from parent components down to their children.
3	**State** Internal data owned by a component. When state changes, React automatically triggers a re-render of that component.
4	**Hooks** Functions like `useState` and `useEffect` that let you manage state and lifecycle in functional components — no class syntax needed.
5	**Modules** A way to group related components together into a cohesive feature or page — keeping codebases organised and scalable.

Component Behaviour

State Drives the UI

In React, the UI is always a **reflection of state**. You never manually manipulate the DOM — instead, you update state and React takes care of the rest.

> When state changes, React compares the new virtual DOM with the previous one and updates **only the affected parts** — a process called reconciliation.

Efficient Rendering

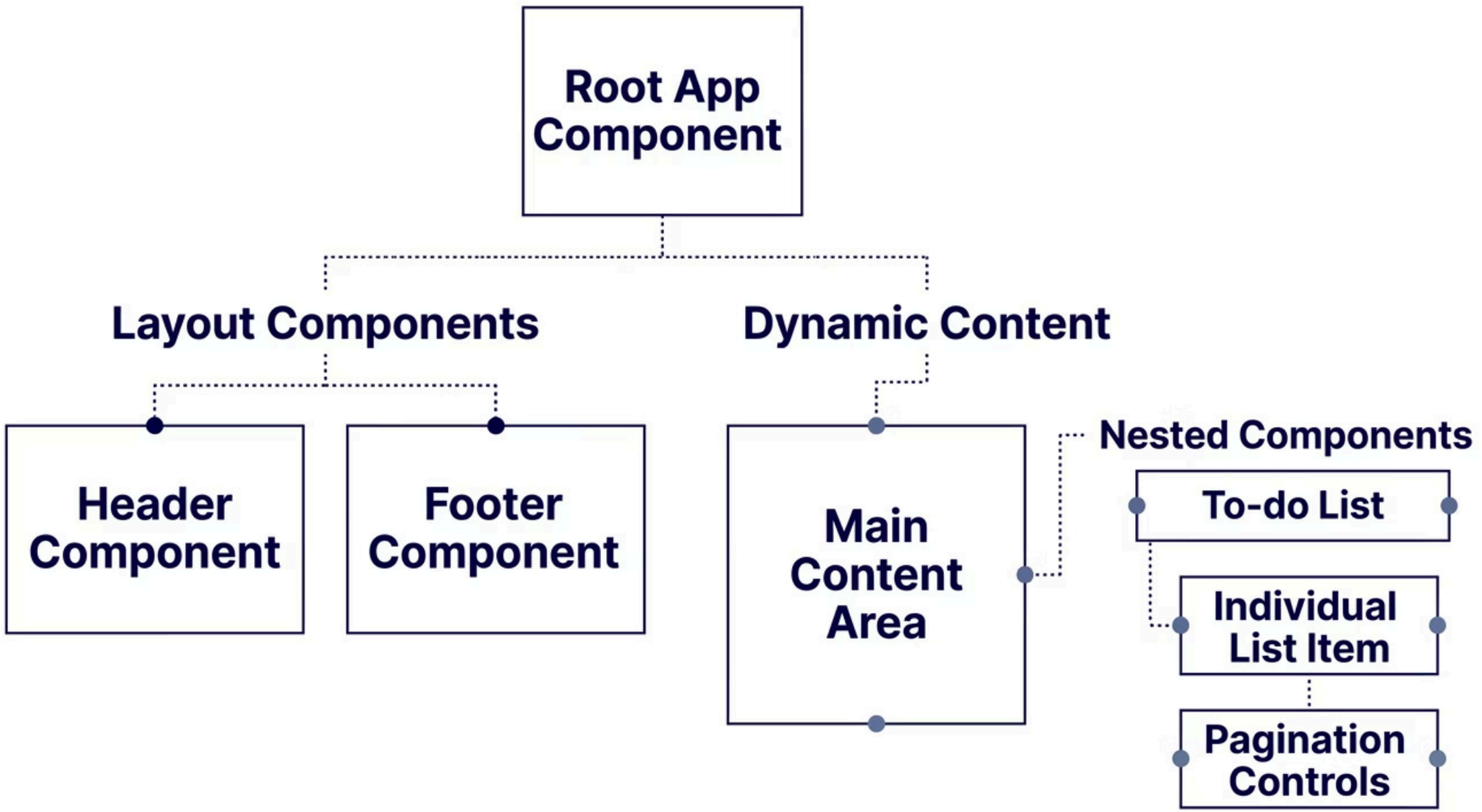
This targeted update strategy means React avoids costly full-page reloads. Only the components whose state or props have changed are re-rendered, keeping applications fast even as they grow in complexity.

The Render Cycle

1. Component initialises with default state
2. User interaction or data fetch triggers a state change
3. React schedules a re-render of the affected component
4. Virtual DOM diff is calculated
5. Only changed elements are updated in the real DOM

Example UI Structure

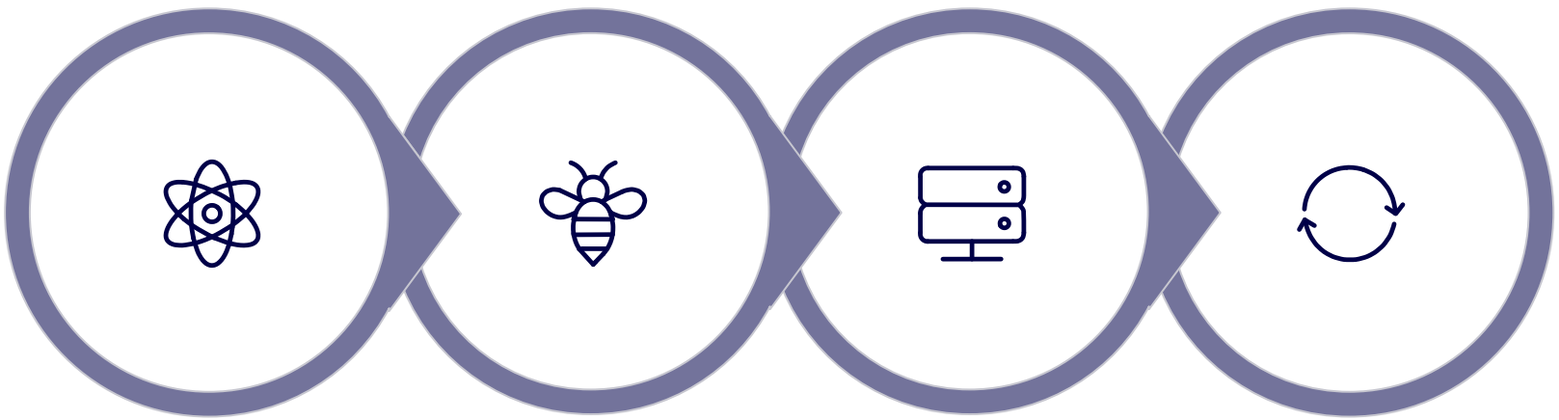
A typical React application is composed of shared layout components and dynamic content components — all nested within each other to form a complete interface.



Shared components like `Header` and `Footer` are rendered once and reused across every page. The main content area swaps dynamically based on routing or user interaction, while deeply nested components — such as a to-do list containing items and pagination — demonstrate React's composability at its finest.

Backend Integration

React is a front-end library — it relies on a backend to persist data, authenticate users, and perform business logic. Communication happens via **REST APIs over HTTP**.



Typical Stack

- React front end (browser)
- REST API layer (HTTP/JSON)
- Spring Boot or Node.js backend
- Relational or NoSQL database

Data Flow

- User interacts with the UI
- React fires an API request
- Backend processes and responds
- React updates state → UI re-renders

Project Structure

A React project is made up of multiple file types, each serving a distinct purpose. Understanding the structure helps teams collaborate and maintain large codebases effectively.

Components (.js / .jsx)

The core of any React app — each component lives in its own JavaScript file, encapsulating its template, logic, and sometimes styles.

HTML via JSX

JSX is a syntax extension that lets you write HTML-like markup directly inside JavaScript — React transforms it into real DOM elements at build time.

CSS Styles

Styles can be global, module-scoped, or component-level — giving teams flexibility in how they manage visual design across the application.

Config & Assets

Configuration files, environment variables, images, fonts, and other static assets are organised alongside the component tree for easy access.

SUMMARY

Why React Matters

React has become the industry standard for front-end development — and for good reason. It solves real problems at scale.

Simplifies Complex UI

Reusable components make even the most complex interfaces maintainable — write once, use everywhere, update in one place.

Efficient Updates

React's virtual DOM ensures only the necessary parts of the UI are updated — no full page reloads, no wasted renders.

Scales for Production

Used by Facebook, Netflix, Airbnb, and thousands of teams worldwide — React is proven to scale for large applications and large engineering teams alike.

> React doesn't just make building UIs easier — it makes them **faster, more maintainable, and ready for the demands of modern web development**.